

# Portfolio Tracker — System Documentation

Personal iPhone app for tracking a multi-currency stock portfolio with live prices. All portfolio data stays on the device — nothing is uploaded to any server. Version is set in `pwa/version.js` (`APP_VERSION` / `APP_BUILD`) — the single source of truth. Everything else derives from it (\$26.6).

Document status: Active

Version: 1.5

Date created: 2026-06-14

Last updated: 2026-10-03

Prepared by: Doug + Claude Code Opus 4.6

Personal iPhone app for tracking a multi-currency stock portfolio with live prices.

All portfolio data stays on the device — nothing is uploaded to any server.

## How to read this document

This document serves two audiences. Use the table below to find the sections most relevant to you.

| If you are...                                                      | Read these sections                                          | You can skip                  |
|--------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------|
| <b>A general user</b> (no IT background)                           | 0, 4, 7 (install steps only), 8, 9, 13, 14, 15, 17, Glossary | 3, 10, 11, 12, 16, 18, 20, 21 |
| <b>An IT person / developer</b> (maintaining or modifying the app) | All sections, especially 10, 11, 12, 16, 18, 20, 21          | —                             |

Sections marked with a phone icon (📱) are written for general users. Sections marked with a wrench icon (🔧) are written for developers. Unmarked sections are relevant to both.

## Table of Contents

- 0. What Does This App Do? 📱
- 1. Document Control
- 2. System Overview

3. System Identity
4. Purpose and Intended Use
5. Builder / Build History
6. Hardware Prerequisites
7. Software Prerequisites
8. Capabilities
9. Limitations
10. Architecture Overview ■
11. Dependencies ■
12. File and Folder Locations ■
13. Core Workflows
14. End-User Operating Guide ■
15. Data Storage and Privacy
16. Housekeeping and Maintenance
17. Troubleshooting
18. Lessons Learned ■
19. Change Log
20. API Reference ■
21. Code Map ■
22. How Updates Reach Your Phone
23. ICC Integration (Investor Command Center) ■
24. iPhone PWA Compatibility Checklist ■
25. Glossary
26. Public Repository & Data-Privacy Procedure ■

## 0. What Does This App Do? ■

Portfolio Tracker is an app that lives on your iPhone home screen. It shows you how your stocks are doing — their current price, how much you've gained or lost since you bought them, and how they moved today.

### **Here's how it works in practice:**

- You open the app and tap Refresh.
- It pulls the latest stock prices from the internet (takes a few seconds).
- You see your total portfolio value, plus each stock's profit or loss.
- You can tap "Today" to see just today's changes, or sort your stocks different ways.

### **What makes it different from other apps:**

- There is no account, no login, no subscription, no ads.
- Your portfolio data never leaves your phone — it is stored only in your browser, on your device.
- It's free and runs from a simple web page saved to your home screen.

### **What it does NOT do:**

- It does not buy or sell stocks for you. It only tracks positions you enter manually.
- It does not give investment advice.
- On the home network, it syncs with the desktop server so the ICC dashboard shows the same data. Away from home, your data lives on the phone only.

If you've ever used a spreadsheet to track stocks, think of this as a nicer version of that spreadsheet, with live prices, that lives on your home screen.

## 1. Document Control

| Field          | Value                                        |
|----------------|----------------------------------------------|
| Document title | Portfolio Tracker PWA — System Documentation |
| Version        | 1.5 (build 27)                               |
| Status         | Active                                       |
| Owner          | Doug                                         |
| Last updated   | 2026-10-03                                   |

Version is set in `pwa/version.js` (`APP_VERSION` / `APP_BUILD`) — the single source of truth. Everything else derives from it (§26.6).

| Storage location | `~/Desktop/Claude Summary/Portfolio Tracker/docs/` |

| Live URL | <https://summerbb138.github.io/portfolio-tracker/> |

| GitHub repo | [summerbb138/portfolio-tracker](https://github.com/summerbb138/portfolio-tracker) (public — no personal data in code) |

## 2. System Overview

### Summary

A single-file Progressive Web App (PWA) that tracks a personal multi-currency stock portfolio on iPhone. The app fetches live stock prices from Yahoo Finance and live exchange rates from `open.er-api.com`, calculates portfolio value in USD, and displays profit/loss from cost basis and daily changes. All data is stored locally in the browser — no server-side processing or data uploads.

### Executive snapshot

| Item            | Summary                                                                                |
|-----------------|----------------------------------------------------------------------------------------|
| System name     | Portfolio Tracker PWA                                                                  |
| Main purpose    | Track personal stock portfolio with live prices across multiple markets and currencies |
| Primary user(s) | Single user (Doug) on iPhone                                                           |

| Item                 | Summary                                |
|----------------------|----------------------------------------|
| Primary interface(s) | iPhone home screen PWA (Safari-based)  |
| Current status       | Operational — deployed on GitHub Pages |

### 3. System Identity ■

| Field                  | Details                                       |
|------------------------|-----------------------------------------------|
| System name            | Portfolio Tracker PWA                         |
| Short name             | Portfolio                                     |
| Project folder         | ~/Desktop/Claude Summary/Portfolio Tracker/   |
| Runtime environment    | Browser (Safari on iPhone, Chrome on desktop) |
| Main language(s)       | HTML, CSS, JavaScript (single file)           |
| Primary execution mode | Static PWA hosted on GitHub Pages             |

## 4. Purpose and Intended Use

### Purpose

Provide a lightweight, permanent, always-available portfolio tracker on iPhone without requiring any paid apps, subscriptions, or account creation. Designed for a multi-market portfolio spanning US, Japan, Hong Kong, UK, Europe, Canada, Australia, and China.

### Intended use cases

- Check current portfolio value and P&L at a glance
- Track daily price changes across all positions
- Buy/sell shares and track cash balances per currency
- Sort and compare positions by value, P&L, or daily performance
- Export/import portfolio data as JSON backup

### Out-of-scope / non-goals

- Trade execution — the app does not place orders
- Investment advice or recommendations
- Real-time streaming quotes (prices are snapshot-on-refresh)
- Multi-user or shared portfolio features
- Cloud sync (local network sync to ICC is supported — see Section 23)

## 5. Builder / Build History

| Question                     | Details                                                                   |
|------------------------------|---------------------------------------------------------------------------|
| Who built the system?        | Doug + Claude Code                                                        |
| When was it built?           | 2026-06-14 to 2026-06-18                                                  |
| How was it built?            | Claude Code Opus 4.6                                                      |
| What was the build approach? | Iterative single-file PWA, feature-by-feature over 3 days                 |
| Subsequent modifications     | Multi-currency cash, daily change tab, sort, persistence (see Change Log) |

### Build narrative

The app was built over four sessions. Day 1 (Jun 14): core portfolio tracking with live prices, position management, and iOS dark-theme styling. Day 2 (Jun 15): multi-currency cash tracking, buy/sell workflows, PWA installation (service worker, manifest, GitHub Pages deployment). Day 3 (Jun 16): daily change tab, structured detail rows, sort control, IndexedDB backup for data persistence, brightness improvements for legibility. Day 4 (Jun 18): portfolio history chart with bar graph, manual gap-filling from Yahoo historical data, UI polish (standardized font sizes with 14px minimum, unified button styling, lighter background, instant tap feedback, active tab indicator).

## 6. Hardware Prerequisites

| Component | Minimum requirement              | Recommended      | Notes                                            |
|-----------|----------------------------------|------------------|--------------------------------------------------|
| Device    | Any device with a modern browser | iPhone 12+       | Optimized for iPhone form factor                 |
| Network   | Internet connection              | WiFi or cellular | Required to fetch live prices and exchange rates |
| Storage   | Negligible (<1 MB)               | —                | Data stored in browser localStorage + IndexedDB  |

## 7. Software Prerequisites

| Component | Required version        | Purpose                      | Notes                                                  |
|-----------|-------------------------|------------------------------|--------------------------------------------------------|
| Browser   | Safari 15+ / Chrome 90+ | Run the PWA                  | Safari required for iPhone home screen install         |
| Python    | 3.x (optional)          | Local dev server (server.py) | Not needed for production — GitHub Pages hosts the app |

## How to install on your iPhone ■

No app store needed. Open a web page and save it to your home screen:

1. Open Safari on your iPhone (it must be Safari — Chrome won't work for this step)
2. Go to: **<https://summerbb138.github.io/portfolio-tracker/>**
3. Tap the **Share button** (the square with an upward arrow, at the bottom of the screen)
4. Scroll down and tap **"Add to Home Screen"**
5. Name it **"Portfolio"** and tap **Add**

The app now appears as a permanent icon, just like any other app. You never need to reinstall it — when the code is updated, the app updates itself (see Section 22).

## 8. Capabilities

| Capability             | What it means                                                                          | How to use it                              |
|------------------------|----------------------------------------------------------------------------------------|--------------------------------------------|
| Live stock prices      | The app pulls the latest price for each stock you own                                  | Tap the Refresh button                     |
| Live exchange rates    | Converts foreign currencies to USD automatically                                       | Happens automatically when you Refresh     |
| Multi-currency support | Tracks stocks in 8 markets: US, Japan, Hong Kong, UK, Europe, Canada, Australia, China | Pick the market when adding a stock        |
| Total value in USD     | Shows your entire portfolio value in one number                                        | Displayed in the summary card at the top   |
| P&L from cost          | Shows how much you've gained or lost vs. what you paid                                 | Default view ("From cost" tab)             |
| Daily change           | Shows how much each stock moved today                                                  | Tap the "Today" tab                        |
| Sort positions         | Organize stocks by name, value, profit, or daily change                                | Tap the sort button in the Holdings header |
| Buy more shares        | Record additional purchases with automatic cost averaging                              | Long-press a stock → Buy More Shares       |
| Sell shares            | Record sales with profit/loss preview                                                  | Long-press a stock → Sell Shares           |
| Multi-currency cash    | Track cash balances in each currency separately                                        | More menu (■) → Set Cash Balance           |
| Export/import          | Save all your data to a file, or restore from a file                                   | More menu (■) → Export / Import            |
| Auto-backup            | Keeps a safety copy of your data inside the browser                                    | Automatic — no action needed               |

| Capability          | What it means                                                                                                                                                                                                                                                                      | How to use it                          |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Restore last backup | One-tap restore if data is lost                                                                                                                                                                                                                                                    | More menu (■) → Restore Last Backup    |
| Portfolio history   | Bar chart of daily portfolio value over time                                                                                                                                                                                                                                       | Tap the "History" tab                  |
| Fill gaps           | Backfill missing weekdays between the last recorded date and yesterday using Yahoo Finance historical daily closes for each position, combined with current exchange rates and cash balances. Only fills forward from existing data — does not backfill before the first snapshot. | History tab → Fill Gaps button         |
| Works offline       | The app itself loads even without internet                                                                                                                                                                                                                                         | Prices won't update, but the app opens |

## 9. Limitations

| What you might notice                          | Why it happens                                                   | What to do                                                                  |
|------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Prices don't update automatically              | The app only fetches prices when you tap Refresh                 | Tap Refresh whenever you want current prices                                |
| Prices might fail to load occasionally         | The relay that helps fetch prices (Cloudflare Worker) can hiccup | Wait a minute and tap Refresh again                                         |
| Data could disappear after a Safari update     | iPhone sometimes clears browser storage                          | The app auto-restores from backup; also use Export regularly                |
| Data is on one phone only                      | There's no cloud sync                                            | Use Export/Import to move data between devices                              |
| Cash can go negative                           | The app doesn't block you from buying without enough cash        | Adjust cash balance manually via More menu                                  |
| Dividends aren't tracked                       | Not a built-in feature                                           | Manually add to your cash balance when dividends arrive                     |
| iPhone might show an old version               | The app caches itself for speed                                  | Refresh the page twice after an update (see Section 22)                     |
| History backfill assumes constant positions    | Fill Gaps uses your current holdings for all past dates          | Stock splits, mergers, or past trades won't be reflected in backfilled data |
| Backfilled history uses current exchange rates | Historical FX rates aren't freely available                      | Minor inaccuracy in USD conversion for past dates                           |
| Dividends not reflected in history             | Cash from dividends isn't backfilled                             | Manually adjust cash; history self-corrects going forward                   |

## 10. Architecture Overview ■

### High-level architecture

Single HTML file containing all CSS and JavaScript. No build tools, no frameworks, no server-side code. The file is served as a static asset from GitHub Pages. All data storage uses browser-native APIs (localStorage and IndexedDB). Price data is fetched client-side from public APIs.

### Architecture table

| Layer / component      | Function                       | Notes                                           |
|------------------------|--------------------------------|-------------------------------------------------|
| pwa/index.html         | Complete PWA (HTML + CSS + JS) | Single file, ~1800 lines                        |
| localStorage           | Primary data store             | Per-origin, volatile on iOS                     |
| IndexedDB              | Backup data store              | More durable than localStorage                  |
| Yahoo Finance API      | Stock price data               | Via personal Cloudflare Worker relay (build 27) |
| open.er-api.com        | Exchange rate data             | Free, CORS-friendly, daily                      |
| Service worker (sw.js) | Offline caching                | Version-bumped on each deploy                   |
| GitHub Pages           | Static hosting                 | Free, permanent URL                             |

### Data flow

1. User opens PWA → service worker serves cached HTML (or fetches from GitHub Pages)
2. App loads positions/cash from localStorage; if empty, checks IndexedDB backup
3. On Refresh: fetches exchange rates from er-api.com, then stock prices sequentially from Yahoo Finance via the Cloudflare relay
4. Prices and rates saved to localStorage + IndexedDB; UI re-renders with updated values
5. On buy/sell/edit: positions and cash updated in both localStorage and IndexedDB

### Design decisions and rationale

Understanding *why* the app is built this way is critical for safe future changes. Each decision below has a constraint behind it — don't change the approach without understanding the constraint first.

| Decision                                               | Why                                                                                                                    | Constraint                                                                     |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Single HTML file (no framework, no build tools)</b> | Simplicity — one file to edit, deploy, and cache. No toolchain to install or break. Easy for Claude Code to work with. | PWA must be servable as a static file from GitHub Pages with zero build steps. |

| Decision                                       | Why                                                                                                                                                                                                                                                                 | Constraint                                                                                                          |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>localStorage + IndexedDB (dual storage)</b> | localStorage is the primary store because it's synchronous and simple. IndexedDB is the backup because iOS Safari can silently purge localStorage under storage pressure (this actually happened — cash balances were lost). Dual-write means data survives purges. | iOS Safari's aggressive storage eviction is unpredictable. A single store is unreliable on iPhone.                  |
| <b>Sequential API calls (not parallel)</b>     | One ticker at a time: gentler on Yahoo and the relay, and parallel fetches caused failures on mobile. Sequential is slower but reliable.                                                                                                                            | Free-tier quotas and mobile networks reward restraint. We control the relay, but requests still terminate at Yahoo. |
| <b>CORS relay instead of a backend</b>         | A backend would require hosting, maintenance, and a domain. Free public CORS proxies proved unreliable (both died in 2026, see §20), so the relay is now our own Cloudflare Worker — still zero server cost and zero ops, but under our control.                    | No-cost constraint: the entire system must run for free.                                                            |
| <b>Service worker for caching</b>              | Makes the app load instantly (from cache) and work offline. But requires manual version bumps — if you forget to bump sw.js, iPhones will serve stale code indefinitely.                                                                                            | GitHub Pages has no server-side cache control. The SW is the only caching mechanism.                                |
| <b>Yahoo Finance v8 chart API</b>              | Provides both current price and previous close in a single call. No API key required. Available globally.                                                                                                                                                           | The exact endpoint and response fields are undocumented by Yahoo — see Section 19 for what we use.                  |
| <b>No authentication or user accounts</b>      | Single-user tool. Adding auth would require a backend, breaking the zero-cost static hosting model.                                                                                                                                                                 | Privacy requirement: no data leaves the device.                                                                     |

## 11. Dependencies ■

| Dependency                                                                       | Type                                          | Why needed                                                                                                   | Failure impact                                          | Where used in code                     |
|----------------------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|----------------------------------------|
| Cloudflare Worker relay ( <code>portfolio-relay.summerbb138.workers.dev</code> ) | Self-hosted CORS relay (Cloudflare free tier) | Yahoo Finance is not CORS-enabled; the relay forwards chart requests and adds CORS headers for this app only | Prices won't update; last-fetched prices remain visible | <code>fetchPrice()</code> — line ~1050 |

| Dependency                 | Type    | Why needed                     | Failure impact                                             | Where used in code                             |
|----------------------------|---------|--------------------------------|------------------------------------------------------------|------------------------------------------------|
| open.er-api.com            | Web API | Free USD exchange rates        | Currency conversion uses stale rates                       | <code>fetchExchangeRates()</code> — line ~1029 |
| Yahoo Finance v8 chart API | Web API | Stock prices + previous close  | Same as relay failure                                      | <code>fetchPrice()</code> — line ~1050         |
| GitHub Pages               | Hosting | Static file hosting with HTTPS | App won't load on first visit (cached version still works) | Deployment target                              |

**No npm packages, no Python libraries, no build dependencies for the production app.**

If any dependency disappears permanently, see Section 19 (API Reference) for what to replace and how.

## 12. File and Folder Locations ■

| Path                         | Purpose                                                  | Deploy to GitHub? | Notes                                                              |
|------------------------------|----------------------------------------------------------|-------------------|--------------------------------------------------------------------|
| pwa/index.html               | Complete PWA application                                 | Yes               | All HTML, CSS, and JS in one file                                  |
| pwa/manifest.json            | PWA manifest for home screen install                     | Yes               | App name, icons, display mode                                      |
| pwa/sw.js                    | Service worker for offline caching                       | Yes               | Bump <code>CACHE_NAME</code> version on each deploy                |
| pwa/icon-192.png             | PWA icon (192x192)                                       | Yes               | Shown on home screen                                               |
| pwa/icon-512.png             | PWA icon (512x512)                                       | Yes               | Splash screen                                                      |
| start.sh                     | Standard launch script                                   | No                | <code>bash start.sh</code> to start local server                   |
| pwa/server.py                | Local development server (port 8091)                     | No                | <code>python3 pwa/server.py</code> to test locally                 |
| pwa/setup.html               | Helper to inject positions via <code>localStorage</code> | No                | For local dev only                                                 |
| data/my-portfolio.json       | Portfolio data for local import                          | No (gitignored)   | Contains personal data                                             |
| docs/PROGRESS.md             | Development progress notes                               | No (gitignored)   | Build history and notes                                            |
| docs/SYSTEM_DOCUMENTATION.md | This document                                            | Yes (public repo) | Keep in <code>docs/</code> folder; Markdown is the source of truth |

| Path                                                             | Purpose                                | Deploy to GitHub? | Notes                                                                                                                   |
|------------------------------------------------------------------|----------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------|
| docs/SYSTEM_DOCUMENTATION.pdf                                    | PDF of this document                   | Yes               | Regenerate on every change via the shared Project Template/docs/md_to_pdf.py (the bespoke generate_pdf.py was retired). |
| pwa/version.js                                                   | Single source of truth for the version | Yes               | APP_VERSION + APP_BUILD; read by sw.js, index.html, server.py, and the ICC. Bump this only.                             |
| scripts/githooks/pre-push                                        | Pre-push data/version guard            | Yes               | Activate per clone: git config core.hooksPath scripts/githooks.                                                         |
| ~/Library/Application Support/PortfolioTracker/my-portfolio.json | Live portfolio data                    | No (outside repo) | Real data — outside the git tree by design (§26). Rotating backups in data/backups/.                                    |

## Key files to know

- **pwa/index.html** — the entire app. Edit this to change any feature. See Section 20 (Code Map) for a guide to what's where inside this file.
- **pwa/sw.js** — bump the cache version number (line 1: portfolio-vN) after **every** code change. If you forget, iPhones will keep serving the old version.
- **data/my-portfolio.json** — your actual portfolio data for import (gitignored, never pushed to GitHub).

## 13. Core Workflows

| What you want to do | Steps                                                                   | What happens                                   |
|---------------------|-------------------------------------------------------------------------|------------------------------------------------|
| See your portfolio  | Open app → tap Refresh                                                  | Summary + all positions with live prices       |
| See today's changes | Tap the "Today" tab                                                     | Daily \$ and % change for each stock and total |
| Sort your stocks    | Tap the sort button in the Holdings header                              | Stocks reorder by your chosen criterion        |
| Add a new stock     | Tap "+ Add Stock" → pick market → fill in ticker, cost, quantity → Save | Stock appears in your list                     |
| Edit a stock        | Tap any stock → change fields → Save                                    | Position updated                               |

| What you want to do | Steps                                                                | What happens                               |
|---------------------|----------------------------------------------------------------------|--------------------------------------------|
| Buy more of a stock | Long-press stock → Buy More Shares → enter price and quantity → Save | Average cost recalculated, cash deducted   |
| Sell some shares    | Long-press stock → Sell Shares → enter price and quantity → Sell     | P&L shown, proceeds added to cash          |
| Set your cash       | More (■) → Set Cash Balance → enter amounts → Save                   | Cash rows updated                          |
| Back up your data   | More (■) → Export Portfolio                                          | JSON file saved to your phone              |
| Restore from file   | More (■) → Import Portfolio → pick the file                          | Portfolio restored                         |
| Quick restore       | More (■) → Restore Last Backup                                       | Restored from last export (no file needed) |

## 14. End-User Operating Guide ■

### Daily use

1. **Open the app** — tap the Portfolio icon on your iPhone home screen
2. **Refresh prices** — tap the Refresh button at the bottom. Prices update one by one (~2 seconds per stock). A message at the top tells you how many prices changed.
3. **Check P&L** — the "From Cost" tab (shown by default, highlighted in darker blue) tells you how much you've made or lost on each stock compared to what you paid
4. **Check daily changes** — tap the "Today" tab to see how much each stock moved since yesterday's close
5. **Sort positions** — tap the sort button (e.g., "Ticker A↓Z") in the Holdings header. Each tap cycles to the next sort: by name, by value, by profit, or by daily change (ascending or descending)

### Managing your stocks

#### Adding a stock:

Tap "+ Add Stock" at the bottom of the list. Pick the market (US, Japan, etc.), then type the ticker symbol, the price you paid per share, and how many shares you own. Tap Save.

#### Editing a stock:

Tap any stock to open its edit screen. Change whatever you need and tap Save.

#### Buying more shares:

Long-press (press and hold) a stock → tap "Buy More Shares." Enter the price you paid and how many shares you bought. The app calculates your new average cost and deducts from your cash balance.

#### Selling shares:

Long-press a stock → tap "Sell Shares." Enter your sell price and the number of shares to sell. The app shows you the profit or loss before you confirm. Tap Sell. The proceeds are added to your cash balance.

#### Deleting a stock:

Tap a stock → scroll down → "Delete Position." The cost basis is returned to your cash balance.

#### Reordering stocks:

Long-press a stock → Move Up or Move Down.

### Managing cash

- Tap a cash row in the summary card, or go to More (■) → Set Cash Balance
- Enter cash amounts for each currency you hold
- Cash adjusts automatically when you buy (deducted) or sell (added) shares

### Backing up your data

- **Export:** More (■) → Export Portfolio → a file called `portfolio-backup.json` is saved to your Downloads folder (open the Files app on iPhone to find it)
- **Import:** More (■) → Import Portfolio → pick the JSON file from Files
- **Quick restore:** More (■) → Restore Last Backup → restores from the last export you made (stored inside the app, no file needed)

**Recommendation:** Export a backup at least once a week. If anything ever goes wrong, you can always get your data back from the file.

## 15. Data Storage and Privacy

### Where your data lives

All your portfolio data is stored **on your phone, inside the browser**. It never leaves the device.

| Data                                        | Where it's stored               | Sent to any server? |
|---------------------------------------------|---------------------------------|---------------------|
| Your stocks (ticker, cost, quantity, price) | On your phone (browser storage) | No                  |
| Your cash balances                          | On your phone (browser storage) | No                  |
| Exchange rates                              | On your phone (browser storage) | No                  |
| Your last export backup                     | On your phone (browser storage) | No                  |

### What the app sends over the internet

The app makes three types of requests — all are to fetch *public* data. Your portfolio is never included.

| Request      | Where it goes                        | What it sends                         | What it gets back |
|--------------|--------------------------------------|---------------------------------------|-------------------|
| Stock prices | Yahoo Finance (via a helper service) | Just the ticker symbol (e.g., "AAPL") | The current price |

| Request        | Where it goes   | What it sends | What it gets back      |
|----------------|-----------------|---------------|------------------------|
| Exchange rates | open.er-api.com | Nothing       | Today's currency rates |
| App files      | GitHub Pages    | Nothing       | The app's code         |

## Privacy summary

- **Your portfolio data is never sent anywhere.** Not to us, not to Yahoo, not to anyone.
- There is no account, no login, no password.
- There are no cookies, no analytics, no tracking, no ads.
- The app's code is public on GitHub, but your stocks and cash are stored only on your phone.
- If you clear Safari's data, your browser storage is erased — that's why Export backups matter.

## 16. Housekeeping and Maintenance ■

### Routine tasks

| Task             | When                              | How                                                                                         | Why                                                         |
|------------------|-----------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Export backup    | Weekly or after big trades        | More → Export Portfolio                                                                     | Protects against iOS clearing browser storage               |
| Refresh prices   | Daily or as needed                | Tap Refresh                                                                                 | Prices don't auto-update                                    |
| Bump SW cache    | After every code change           | Edit sw.js line 1: change portfolio-vN to portfolio-v(N+1)                                  | Without this, phones keep serving the old code              |
| Push to GitHub   | After editing index.html or sw.js | git add index.html<br>sw.js && git commit<br>-m "description" &&<br>git push origin<br>main | Deploys to the live URL                                     |
| Verify on iPhone | After pushing                     | Open PWA → refresh<br>twice → verify changes<br>appear                                      | First refresh loads new service worker; second activates it |

### Developer deployment checklist

1. Make your changes to index.html
2. Open sw.js and bump the version number (e.g., portfolio-v6 → portfolio-v7)
3. Test locally: `bash start.sh` → open `http://localhost:8091`
4. Commit both files: `git add index.html sw.js && git commit -m "what changed"`
5. Push: `git push origin main`
6. Wait ~30 seconds for GitHub Pages to deploy
7. On iPhone: open the PWA, pull to refresh or tap Refresh, then close and reopen the app

8. Verify the change is live

## Recommended housekeeping

- Export a backup at least once a week
- After pushing changes, always bump sw.js — this is the #1 source of "why isn't my change showing up?"
- Periodically verify your cash balances match your actual brokerage accounts

## 17. Troubleshooting

### For general users ■

| What you see                                                 | What it means                                                                         | What to do                                                                                                                                                                                   | Is my data safe?                                        |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Prices say "loading" and don't change                        | The internet price service is slow or busy                                            | Wait a minute, then tap Refresh again                                                                                                                                                        | Yes — your stocks are fine, just prices aren't updating |
| Prices don't change even though "Prices are current" appears | The CORS proxy or Safari itself is serving cached (stale) data instead of live prices | Fixed in build 10 (URL cache-busting) and build 12 (cache : 'no-store' on all fetches). If it recurs, check both fixes are present in <code>fetchJSON()</code> and <code>fetchPrice()</code> | Yes                                                     |
| All my stocks are gone                                       | Your phone cleared the app's storage                                                  | Tap More (■) → Restore Last Backup. If that doesn't work, use Import with your backup file                                                                                                   | If you exported recently, yes                           |
| My cash balances disappeared but stocks are there            | Partial storage clearing (a known iPhone issue)                                       | Tap More (■) → Restore Last Backup. Fixed in build 13: import/restore now always includes cash balances                                                                                      | Yes — the backup has your cash                          |
| The app looks different from what I expected                 | You might be seeing an older version                                                  | Close the app completely (swipe up in app switcher), then reopen it. If still wrong, open Safari, go to the URL, and refresh the page twice                                                  | Yes — display issues don't affect data                  |
| Export doesn't seem to save anything                         | Safari might be blocking the download                                                 | Try opening the URL in Safari (not the home screen app) and exporting from there                                                                                                             | Yes — your data is still in the app                     |
| A Japanese stock shows the wrong price                       | The ticker might have the wrong market suffix                                         | Tap the stock → check that the market is set to "Japan"                                                                                                                                      | Yes                                                     |

## For developers ■

| Problem                                              | Likely cause                                                                   | How to fix                                                                                                                                                                                                                                                      |
|------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Changes not appearing on iPhone                      | sw.js cache version not bumped                                                 | Bump version, push, user refreshes twice                                                                                                                                                                                                                        |
| fetchPrice() returning null                          | Relay or Yahoo unavailable, or Yahoo API change                                | Check browser console; see §20 for relay/API details                                                                                                                                                                                                            |
| Prices "succeed" but are stale                       | CORS proxy caching Yahoo responses, or Safari HTTP cache serving old responses | Two layers of defence: (1) <code>&amp;t=\${Date.now() }</code> cache-buster on the Yahoo URL, (2) <code>cache: 'no-store'</code> in the fetch options. Both are needed — the URL trick defeats the proxy cache, the fetch option defeats Safari's browser cache |
| Exchange rates stale                                 | er-api.com response cached in localStorage                                     | Tap Refresh, or manually clear <code>portfolio_rates</code> in console                                                                                                                                                                                          |
| IndexedDB restore not working                        | Schema version mismatch after code change                                      | Check <code>IDB_VERSION</code> constant and <code>openIDB()</code> upgrade handler                                                                                                                                                                              |
| Blank screen on load                                 | JavaScript syntax error                                                        | Open browser console (Cmd+Option+J in Chrome); fix syntax error                                                                                                                                                                                                 |
| <code>restoreFromIDB()</code> runs but data is empty | Backup was never written (first-time user)                                     | Normal — IndexedDB backup is created on first <code>save()</code> call                                                                                                                                                                                          |

## Emergency recovery

If the app is completely broken after a code push:

1. Check GitHub Pages status at [github.com/summerbb138/portfolio-tracker](https://github.com/summerbb138/portfolio-tracker)
2. Open browser console and look for JavaScript errors
3. If needed, revert: `git revert HEAD && git push origin main`
4. **Portfolio data is safe in IndexedDB on the device** regardless of code problems — the data and the code are stored separately

## 18. Lessons Learned ■

Practical insights from building and iterating on this app, captured to prevent repeating mistakes.

### Minimum font size for mobile readability

The app originally used 12–13px for many secondary labels (tab buttons, section headers, detail rows, chart range chips). On an iPhone screen, these were noticeably hard to read. A floor of **14px** was established for all visible text elements, with the exception of the "Updated" timestamp (13px, since it is informational and rarely read). The summary labels, detail rows, section headers, sort button, chart subtitle, range chips, form labels,

and version string were all bumped to 14px. Tab and toolbar buttons were set at 15px for additional prominence.

**Rule:** No font-size below 14px for any element the user actively reads. Use 13px only for passive/informational text like timestamps.

## Standardized button styling eliminates visual inconsistency

The tab buttons (From Cost, Today, History) and toolbar buttons (Add Stock, Refresh, ■) were originally styled independently — different font sizes, padding, widths, backgrounds, and layout methods. This created visible inconsistency on iPhone: buttons in the same row appeared different widths, and the two rows looked unrelated.

The fix was to make the tab buttons reuse the `.toolbar-btn.primary` CSS class directly, and have both rows use `display: grid; grid-template-columns: repeat(3, 1fr)` with matching gap and padding. Using the same class eliminates drift — any future change to button styling applies to both rows automatically.

**Rule:** When two groups of buttons should look identical, use the same CSS class. Don't create parallel styles that must be kept in sync manually.

## CSS transitions cause perceived lag on mobile

Adding `transition: opacity 0.15s, transform 0.1s` with a `:active` press effect (scale + opacity change) to the tab buttons caused noticeable lag on iPhone. The animation itself was only ~200ms, but combined with the heavy `render()` function that rebuilds the entire DOM on tab switch, the user perceived a delayed response.

Two fixes were applied:

1. **Replaced the animation with an instant color change** — the `:active` pseudo-class now sets `background: #0060c0` (darker blue) with no transition. The visual feedback is immediate.
2. **Deferred the heavy render via `requestAnimationFrame()`** — the button's visual state updates in the current frame, and the DOM rebuild happens in the next frame. This ensures the user sees feedback before the expensive work begins.

**Rule:** On mobile, avoid CSS transitions on elements that trigger heavy JavaScript. Use instant property changes (`background-color`, `box-shadow`) for tap feedback. If a tap triggers expensive DOM work, defer it with `requestAnimationFrame()` so the visual feedback renders first.

## Active tab indicator must be persistent, not transient

The active tab was originally indicated by a subtle inner glow (`box-shadow: inset 0 0 0 1.5px rgba(255, 255, 255, 0.3)`), which was nearly invisible on iPhone. The `:active` press color (`#0060c0`) was also too close to the resting blue (`#0a84ff`) to notice.

The solution was to make the active tab permanently display a darker blue background (`#004494`) — visually distinct from the resting `#0a84ff` at all times, not just during a press. This makes it immediately obvious which tab is selected when the app launches or when switching between tabs.

**Rule:** Active/selected state indicators should be always-visible and high-contrast, not subtle or transient. The user should be able to glance at the screen and instantly know which tab is selected.

## Background color contrast for visual hierarchy

The original pure black background (#000000) made the summary card and holdings list blend into the page. Changing to a lighter grey (#3a3a3c) creates contrast between the page background and the card/list surfaces (#1c1c1e), establishing visual hierarchy and making the content areas stand out.

**Rule:** Use a lighter background behind dark cards to create depth. Pure black backgrounds eliminate contrast with dark surfaces.

## 19. Change Log

| Date       | Version | Change                                                                                                     |
|------------|---------|------------------------------------------------------------------------------------------------------------|
| 2026-06-14 | 1.0     | Core portfolio tracker — live prices, position management, iOS dark theme, cash position, currency symbols |
| 2026-06-15 | 1.1     | Multi-currency cash tracking — buy/sell/delete adjust cash per currency                                    |
| 2026-06-15 | 1.1     | PWA installation — service worker, manifest, icons, GitHub Pages deployment, CORS proxy fallback           |
| 2026-06-15 | 1.2     | Sequential price fetching to avoid CORS proxy rate-limiting                                                |
| 2026-06-16 | 1.3     | Daily change tab ("Today"), structured detail rows (Shares/Cost/Price), sort control (8 modes)             |
| 2026-06-16 | 1.3     | IndexedDB dual-storage backup with auto-restore; Restore Last Backup option                                |
| 2026-06-16 | 1.3     | Brightened all dim labels (--text3: 30% → 60% → 75% opacity); bumped small fonts to 13px                   |
| 2026-06-16 | 1.3     | Tightened spacing between cash position rows                                                               |
| 2026-06-16 | 1.3     | Added fallback CORS proxy (allorigins.win) and fixed silent fetch failure reporting                        |
| 2026-06-16 | 1.3     | Fixed stale prices from proxy caching — added cache-busting &_t=timestamp to Yahoo URL                     |
| 2026-06-16 | 1.3     | Added version display in More menu (build number matches SW cache version)                                 |

| Date       | Version         | Change                                                                                                                                                                                     |
|------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026-06-16 | 1.3             | Fixed Safari browser-level caching of proxy responses — added cache: 'no-store' to all fetch calls; added allorigins to SW bypass list                                                     |
| 2026-06-18 | 1.3             | Fixed backup/restore ignoring cash positions — import now always restores (or clears) cash balances from the backup file; export/import/restore toasts now confirm cash positions included |
| 2026-06-18 | 1.3             | Clear All Holdings now also clears all cash positions                                                                                                                                      |
| 2026-06-18 | 1.4             | Added History tab with daily portfolio value bar chart, manual backfill from Yahoo Finance historical data, range selector (1M/3M/1Y/All), tap-to-inspect tooltips                         |
| 2026-06-18 | 1.4 (build 2)   | Standardized font sizes: 14px minimum across all UI elements (was 12-13px); tab and toolbar buttons set to 15px                                                                            |
| 2026-06-18 | 1.4 (build 3)   | Standardized button sizing: tab bar margins/gap matched to toolbar; background color changed from #000 to lighter grey; "From cost" → "From Cost"                                          |
| 2026-06-18 | 1.4 (build 4-5) | Unified tab buttons with toolbar buttons: both use same CSS class (.toolbar-btn.primary), same grid layout, same sizing                                                                    |
| 2026-06-18 | 1.4 (build 6-7) | Fixed tab button lag: removed CSS transition/animation, replaced with instant darker-blue tap feedback (#0060c0), deferred render() via requestAnimationFrame                              |
| 2026-06-18 | 1.4 (build 8)   | Active tab now permanently displays darker blue (#004494) for clear selection indicator; all toolbar buttons white-on-blue; background settled at #3a3a3c                                  |

| Date       | Version        | Change                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026-07-02 | 1.4 (build 9)  | Standard GET /api/health endpoint added ({status, version, port}); ICC health pings excluded from access logs. Project under local git version control (commit after changes: <code>git add -A &amp;&amp; git commit -m "..."</code> )                                                                                                                                                                                                                                                                                                                                                                               |
| 2026-08-04 | 1.5 (build 25) | <b>Cost now includes cash balances</b> (was stock cost only), consistent with Value — which already included cash. P&L is unchanged (cash appears on both sides and cancels); Return is now measured against total account capital (stock cost + cash), so idle cash dilutes the %. Server /api/portfolio (ICC feeder): <code>totalCost</code> now includes cash, which also fixes a pre-existing P&L overstatement where <code>totalValue</code> included cash but <code>totalCost</code> did not. UI build number realigned to the SW cache version (had drifted at build 8 while <code>sw.js</code> reached v25). |
| 2026-08-04 | 1.5 (build 26) | <b>Foolproofing (§26.6):</b> single version source <code>pwa/version.js</code> ( <code>sw.js/index.html/server.py/ICC</code> all derive from it — bump one file); live data moved OUTSIDE the repo ( <code>~/Library/Application Support/PortfolioTracker/</code> ) so it can't be committed; rotating <code>data/backups/</code> (last 10, gitignored, inside the disaster-recovery tar); committed pre-push hook ( <code>scripts/ghooks/pre-push</code> ) that blocks data-shaped commits. Doc PDF now via shared <code>md_to_pdf.py</code> .                                                                      |

| Date       | Version        | Change                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026-10-03 | 1.5 (build 27) | <b>Quotes moved to a self-hosted Cloudflare Worker relay</b> after the free public proxies died (details in §20): corsproxy.io dropped anonymous access ~2026-09-13 (403 keyless_legacy_url) and api.allorigins.win went down (522), so the iPhone app stopped updating on 2026-09-13. CORS_PROXIES is now a single entry at <a href="https://portfolio-relay.summerbb138.workers.dev">https://portfolio-relay.summerbb138.workers.dev</a> (forwards Yahoo chart URLs only, origin-restricted, never cached); both service workers bypass the relay host. Relay + app path verified end-to-end (US/CA/JP tickers + iPhone refresh). |
| 2026-08-04 | 1.5 (build 25) | <b>Restored public GitHub Pages hosting</b> (no app-code change). The repo had been switched to private, which silently disabled Pages on the free plan and broke iPhone updates. Reset git history to a single clean commit and hardened data privacy so no real portfolio data exists in the repo or its history; added a root <code>index.html</code> redirect to <code>/pwa/</code> (the app had moved to <code>/pwa/</code> , leaving the Pages root URL with nothing to serve); removed the stale generated PDF. Full procedure and future reminders: see §26.                                                                |

## 20. API Reference ■

This section documents the exact external API calls the app makes. If any service changes or goes offline, this is what you need to replace.

### Stock prices — Yahoo Finance v8 chart API

**URL template:**

```
https://portfolio-relay.summerbb138.workers.dev/?url=https://query1.finance.yahoo.com/v8/finance/chart/{SYMBOL}?range=1d&interval=1d&t={TIMESTAMP}
```

The `&t={TIMESTAMP}` parameter is a cache-buster (set to `Date.now()`). The relay itself never caches, but the parameter is kept — historically (public-proxy era) a missing cache-buster caused prices to appear to update while actually showing yesterday's close, and it remains a guard against any upstream/browser caching.

## SYMBOL format by market:

| Market    | Suffix           | Example   |
|-----------|------------------|-----------|
| US        | (none)           | AAPL      |
| Japan     | .T               | 7203.T    |
| Hong Kong | .HK              | 0005.HK   |
| UK        | .L               | SHEL.L    |
| Europe    | .PA / .DE / etc. | MC.PA     |
| Canada    | .TO              | SHOP.TO   |
| Australia | .AX              | BHP.AX    |
| China     | .SS / .SZ        | 600519.SS |

## Response fields used:

```
response.chart.result[0].meta.regularMarketPrice → current price
response.chart.result[0].meta.chartPreviousClose → yesterday's closing price
response.chart.result[0].meta.previousClose → fallback for previous close
response.chart.result[0].meta.currency → currency code (USD, JPY, etc.)
```

**Failure behavior:** If the fetch fails or times out (10-second timeout), the stock's price is left unchanged from the last successful fetch. No error is shown to the user — the "Updated" timestamp reveals staleness.

**Rate limiting:** The relay runs on Cloudflare's free tier (100,000 requests/day — far above this app's needs). Prices are fetched sequentially (one at a time) with no delay between them, which keeps Yahoo happy too.

## Exchange rates — open.er-api.com

### URL:

```
https://open.er-api.com/v6/latest/USD
```

## Response fields used:

```
response.rates.JPY → e.g., 149.5
response.rates.HKD → e.g., 7.82
response.rates.GBP → e.g., 0.79
... (all currencies)
```

**Failure behavior:** If the fetch fails, the app falls back to hardcoded approximate rates defined in `fetchExchangeRates()` (line ~1029). These are rough approximations — the UI still works but currency conversion may be inaccurate.

## CORS relay — personal Cloudflare Worker

**How it works:** The browser can't call Yahoo Finance directly due to CORS restrictions (Yahoo doesn't allow requests from web pages). Since build 27 the app routes Yahoo requests through our own Cloudflare Worker, `portfolio-relay`, defined in the `CORS_PROXIES` array (~line 1189 in `pwa/index.html`). The `fetchViaProxy()` function tries each entry in order — currently just the worker:

```
https://portfolio-relay.summerbb138.workers.dev/?url={URL-encoded Yahoo chart URL}
```

The worker forwards only `query1/query2.finance.yahoo.com /v8/finance/chart/*` URLs, answers CORS only for this app's origin (`https://summerbb138.github.io`) and the local dev servers (`localhost:8091`), refuses all other targets (403), logs nothing, stores nothing, and never caches (`Cache-Control: no-store`). Account: Cloudflare free plan (`summerbb138@gmail.com`), Worker name `portfolio-relay`. Any future browser-based fetcher in the fleet can reuse the same relay account.

**Why we moved (2026-10):** The chain of free public proxies died. `corsproxy.io` dropped anonymous keyless access (~2026-09-13; 403 `keyless_legacy_url`, API key now required) and `api.allorigins.win` went down (Cloudflare 522) — the iPhone app stopped updating on 2026-09-13 and showed only a stale "Updated" timestamp. Public proxies are not production infrastructure; the relay is.

**Critical: two layers of cache defence remain.**

1. **URL parameter:** The `&t=${Date.now() }` cache-buster on the Yahoo URL makes each request unique. (Public proxies cached upstream responses; the relay doesn't, but the buster costs nothing and guards against upstream/browser caches.)
2. **Fetch option:** Safari on iPhone aggressively caches HTTP responses. The cache: 'no-store' option in `fetchJSON( )` forces Safari to bypass its browser cache entirely. Both service workers also bypass the relay host in their fetch handlers so quote JSON is never cached by the SW.

**If the relay fails:** It's our code — check/redeploy the Worker in the Cloudflare dashboard (free plan). As a last resort, add a new entry to `CORS_PROXIES`; each entry maps a URL to a proxied URL.

## 21. Code Map ■

The entire application is in `index.html` (~1800 lines). This map shows where everything lives so you can find what you need quickly.

### File structure

| Line range | Section                     | What's there                                                                             |
|------------|-----------------------------|------------------------------------------------------------------------------------------|
| 1–11       | HTML head                   | Meta tags, manifest link, viewport                                                       |
| 12–278     | <code>&lt;style&gt;</code>  | All CSS — variables, layout, components                                                  |
| 280–510    | HTML body                   | UI structure — nav bar, summary card, tab bar, position list, all bottom sheets, toolbar |
| 511–1330   | <code>&lt;script&gt;</code> | All JavaScript                                                                           |

### CSS landmarks (lines 12–278)

| Lines | What            | Notes                                                                    |
|-------|-----------------|--------------------------------------------------------------------------|
| 13–30 | :root variables | Colors, fonts, spacing. <code>--text3</code> controls dim label opacity. |
| 31–60 | Base styles     | Body, safe-area, scrolling                                               |

| Lines   | What                   | Notes                                     |
|---------|------------------------|-------------------------------------------|
| 61–100  | Nav bar + summary card | .nav, .summary-card,<br>.summary-row      |
| 101–130 | Tab bar + sort         | .tab-bar, .tab, .sort-btn                 |
| 131–200 | Position cards         | .stock-item, .detail-rows,<br>.detail-row |
| 201–260 | Bottom sheets          | .sheet, .sheet-content,<br>market chips   |
| 261–278 | Toolbar + toast        | .toolbar, .toast                          |

## JavaScript landmarks (lines 511–1330)

| Lines   | Function(s)                                                                                                    | Purpose                                                      |
|---------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 512–524 | MARKETS, MARKET_MAP,<br>CURRENCY_SYMBOL, csym( )                                                               | Market definitions and currency<br>formatting                |
| 526–566 | STORAGE_KEY, openIDB( ),<br>idbSet( ), idbGet( ),<br>backupToIDB( )                                            | Storage constants and IndexedDB<br>helpers                   |
| 567–569 | positions, exchangeRates,<br>cashBalances                                                                      | App state — loaded from<br>localStorage on startup           |
| 577–588 | restoreFromIDB( )                                                                                              | Restores all data from IndexedDB if<br>localStorage is empty |
| 590–604 | activeTab, SORT_MODES,<br>sortIndex, actionIndex                                                               | UI state variables                                           |
| 606–608 | save( ), saveCashBalances( ),<br>saveRates( )                                                                  | Persist to localStorage + trigger<br>IndexedDB backup        |
| 609–657 | getCashCurrencies( ),<br>totalCashUSD( ),<br>openCashSheet( ), saveCash( )                                     | Cash balance management                                      |
| 659–670 | buildMarketChips( ),<br>selectMarket( )                                                                        | Market selector UI in Add Stock<br>sheet                     |
| 672–760 | openSheet( ), closeSheet( ),<br>openAddStock( ),<br>openEditStock( ),<br>savePosition( ),<br>deletePosition( ) | Bottom sheet open/close and<br>position CRUD                 |

| Lines     | Function(s)                                                                                                                                                                                                                                                                                   | Purpose                                                                          |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| 762–967   | <code>openPosActions()</code> ,<br><code>addSharesFromAction()</code> ,<br><code>openAddShares()</code> ,<br><code>saveAddShares()</code> ,<br><code>openSellShares()</code> ,<br><code>saveSellShares()</code> , <code>sellAll()</code> ,<br><code>moveUp()</code> , <code>moveDown()</code> | Long-press action menu, buy more, sell, reorder                                  |
| 969–983   | <code>openMoreActions()</code> ,<br><code>switchTab()</code> , <code>cycleSort()</code>                                                                                                                                                                                                       | More menu, tab switching, sort cycling                                           |
| 984–1022  | <code>getSortedIndices()</code>                                                                                                                                                                                                                                                               | Sort logic — returns array of indices sorted by selected mode                    |
| 1023–1094 | <code>fetchJSON()</code> ,<br><code>fetchExchangeRates()</code> ,<br><code>fetchPrice()</code> , <code>refreshAll()</code>                                                                                                                                                                    | All API calls. This is where prices and rates are fetched.                       |
| 1095–1144 | <code>exportData()</code> ,<br><code>restoreLastExport()</code> ,<br><code>importData()</code> ,<br><code>handleImport()</code> , <code>clearAll()</code>                                                                                                                                     | Export/import/clear functionality                                                |
| 1146–1293 | <code>fmt()</code> , <code>fmtUSD()</code> , <code>render()</code>                                                                                                                                                                                                                            | Number formatting and the main render function (rebuilds entire UI)              |
| 1295–1310 | <code>showToast()</code>                                                                                                                                                                                                                                                                      | Toast notification                                                               |
| 1312–1330 | Init block                                                                                                                                                                                                                                                                                    | <code>restoreFromIDB().then(() =&gt; refreshAll())</code> — app startup sequence |

## Key functions to understand before making changes

`render()` (**line ~1156**) — The most important function. Rebuilds the entire UI from state. Uses a two-pass approach: first pass calculates totals, second pass renders HTML. Tab-aware (shows cost P&L or daily change based on `activeTab`). Sort-aware (uses `getSortedIndices()`). If you change the UI, this is where you'll work.

`fetchPrice(index)` (**line ~1050**) — Fetches one stock's price from Yahoo via the Cloudflare relay. Extracts `regularMarketPrice` and `chartPreviousClose` from the response. Called sequentially by `refreshAll()`.

`save()` / `saveCashBalances()` (**lines 606–607**) — Every mutation to positions or cash must call these. They write to `localStorage` AND trigger `backupToIDB()`. If you add a new data type, follow the same pattern.

`restoreFromIDB()` (**line ~577**) — Called once at startup. If `localStorage` is empty, restores positions, cash, and rates from the IndexedDB backup. This is the safety net for iOS storage purges.

## 22. How Updates Reach Your Phone

For general users ■

When the app's code is improved, the update is published to the internet automatically. Here's how it reaches your phone:

1. The new code is uploaded to GitHub (this happens on the developer's computer)
2. GitHub makes it available at the app's web address within about 30 seconds
3. **Next time you open the app**, your phone notices there's a new version and downloads it in the background
4. **The new version takes effect the second time you open the app** — the first opening downloads it, the second opening uses it

**In practice:** just use the app normally. Updates will appear within a day or two of opening it. If you want to force an update immediately, open the app, close it completely (swipe up in the app switcher), and open it again.

**Your data is never affected by updates.** Your stocks and cash are stored separately from the app's code. An update changes how the app looks or works, never what's in your portfolio.

## For developers ■

The update path is: `git push` → GitHub Pages deploy (~30s) → user opens PWA → service worker fetches new files → **next** open activates them.

The service worker (`sw.js`) uses a cache-first strategy. On each load, it serves from cache immediately, then checks for updates in the background. If `CACHE_NAME` has changed (e.g., `portfolio-v6` → `portfolio-v7`), the new SW installs and the old cache is deleted on the next activation.

**Critical:** if you push a code change to `index.html` but forget to bump `CACHE_NAME` in `sw.js`, the service worker sees no change and keeps serving the cached old version indefinitely. Always bump the version.

## 23. ICC Integration (Investor Command Center) ■

### How portfolio data flows to the ICC Overview

The ICC Overview dashboard displays portfolio data from this module. The data flows through a single path:

```
PWA (browser) → save() → localStorage + server file (my-portfolio.json)
↓
ICC Overview → /api/dashboard → Portfolio Tracker /api/portfolio → reads file → fetches live
quotes + FX → returns totals
```

1. The PWA writes positions and cash balances to `data/my-portfolio.json` via `POST /api/portfolio/sync` on every save
2. ICC calls `GET /api/portfolio` on Portfolio Tracker (port 8091), which reads the file, fetches live quotes and exchange rates, and returns computed totals (`totalValue`, `totalCost`, `totalPL`, `todayPL`)
3. ICC displays the result directly — it performs no independent computation

This single computation path guarantees the ICC Overview and Portfolio Tracker always show the same value. Cash balances (all currencies converted to USD) are included in **both** `totalValue` and `totalCost`, so the two figures are consistent. Because cash is on both sides, `totalPL` ( $= \text{totalValue} - \text{totalCost}$ ) reflects stock gains only — cash cancels out. (Prior to v1.5, cash was included in `totalValue` but not `totalCost`, which overstated `totalPL` by the cash balance; this was corrected when Cost was made cash-inclusive.)

### Server-side API endpoints

| Endpoint            | Method | Purpose                                                                                        |
|---------------------|--------|------------------------------------------------------------------------------------------------|
| /api/portfolio      | GET    | Returns fully computed portfolio with live quotes, FX conversion, and totals (consumed by ICC) |
| /api/positions      | GET    | Returns raw positions and cash from my-portfolio.json (consumed by PWA on startup)             |
| /api/portfolio/sync | POST   | Receives {positions, cashBalances} from PWA and writes to my-portfolio.json                    |

## Sync behavior

- **On PWA startup:** if localStorage has positions, pushes them to the server file via /api/portfolio/sync. If localStorage is empty (new device), pulls from the server file via /api/positions.
- **On every save** (add/edit/remove stock, buy/sell, change cash): writes to localStorage, IndexedDB, and server file.
- **iPhone offline:** the PWA works entirely from localStorage. Sync occurs next time the device is on the same network as the server.

## 24. iPhone PWA Compatibility Checklist ■

When modifying Portfolio Tracker code, follow these rules to avoid breaking the iPhone app.

### Must-do on every code change

1. **Edit** pwa/index.html, **not the root** index.html — the server serves from the pwa/ directory. The root copy was removed to prevent this mistake.
2. **Bump** CACHE\_NAME in pwa/sw.js (line 1) — change portfolio-vN to portfolio-v(N+1). Without this, the iPhone PWA keeps serving the old cached code indefinitely.
3. **Push both files** — pwa/index.html and pwa/sw.js must be committed and pushed together.

### Must not break

- **localStorage read/write** — the iPhone PWA relies on localStorage as its primary store. Never change the storage keys (STORAGE\_KEY, CASH\_MULTI\_KEY) without a migration path.
- **IndexedDB backup/restore** — this is the safety net for iOS storage purges. Don't change the schema version without handling the upgrade.
- **Offline functionality** — the PWA must load and display cached data without network access. API calls (/api/\*) are excluded from service worker caching, but the HTML/JS must work offline.
- **Sequential price fetching** — one ticker at a time; don't switch to parallel fetching for client-side quote calls.

### Safe to change

- Server-side code (`pwa/server.py`) — the iPhone PWA doesn't depend on it when accessing via GitHub Pages. Server endpoints are only used when the phone is on the same local network.
- CSS styling — as long as the dark theme and 14px minimum font size are maintained.
- New features — as long as they degrade gracefully when the server is unreachable.

## 25. Glossary

Plain-English definitions of technical terms used in this document.

| Term                                           | What it means                                                                                                                                                                                                                                                                              |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PWA (Progressive Web App)</b>               | A website that can be saved to your phone's home screen and used like a regular app. It's not downloaded from the App Store — you save it from Safari.                                                                                                                                     |
| <b>localStorage</b>                            | A storage area inside your web browser where websites can save small amounts of data. It stays there until you clear your browser data or your phone decides to free up space.                                                                                                             |
| <b>IndexedDB</b>                               | A second, more durable storage area inside your browser. The app uses both localStorage and IndexedDB as a safety measure.                                                                                                                                                                 |
| <b>Service worker</b>                          | A small helper program that runs in the background. In this app, it saves a copy of the app so it loads instantly and works even without internet.                                                                                                                                         |
| <b>CORS proxy</b>                              | A middleman service on the internet. Yahoo Finance doesn't allow web apps to fetch prices directly, so the app sends requests through a middleman that forwards them. Since build 27 this is our own Cloudflare Worker ( <code>portfolio-relay</code> ); before that, free public proxies. |
| <b>GitHub Pages</b>                            | A free service from GitHub that hosts websites. The app's code lives here so anyone with the URL can access it.                                                                                                                                                                            |
| <b>JSON</b>                                    | A standard file format for storing data. When you export your portfolio, it's saved as a JSON file — a plain text file that both humans and computers can read.                                                                                                                            |
| <b>API (Application Programming Interface)</b> | A way for one program to request data from another. The app uses Yahoo Finance's API to ask for stock prices and <code>open.er-api.com</code> 's API to ask for exchange rates.                                                                                                            |
| <b>CORS (Cross-Origin Resource Sharing)</b>    | A security rule in web browsers that prevents websites from talking to other websites unless allowed. Yahoo Finance doesn't allow it, which is why we need the CORS proxy.                                                                                                                 |

| Term                  | What it means                                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Git / GitHub          | Git is a tool for tracking changes to code. GitHub is a website that stores the code and makes it available online. When changes are "pushed" to GitHub, the live app updates. |
| Cache                 | A saved copy of something for faster access. The service worker caches the app so it doesn't need to download from the internet every time you open it.                        |
| Ticker symbol         | The short code for a stock (e.g., AAPL for Apple, 7203.T for Toyota on the Tokyo exchange).                                                                                    |
| P&L (Profit and Loss) | How much money you've made or lost. Shown as both a dollar amount and a percentage.                                                                                            |
| Cost basis            | The price you originally paid per share. Used to calculate your profit or loss.                                                                                                |
| Exchange rate         | How much one currency is worth in another. For example, 1 USD = 149.5 JPY. The app converts all currencies to USD for the total.                                               |

## 26. Public Repository & Data-Privacy Procedure ■

**Critical invariant: this repo is PUBLIC, and real portfolio data must NEVER exist in it — not in tracked files, and not in git history.** The repo is public because GitHub Pages (which delivers the iPhone PWA) only works on *public* repos under the free plan. This section records why, and exactly how we keep real data out, so no one has to reverse-engineer it later.

### 26.0 The three traps we hit — and the guard for each (audit checklist)

If you change anything here, confirm all three guards still hold:

| Trap (what bit us)                                                                                                         | Guard                                                                                                                                                                                                                                           | Where      |
|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>PWA version and ICC-feeder version drift out of sync</b> (the badge said "build 8" while the service worker was at v24) | One version source, <code>pwa/version.js.sw.js</code> , <code>index.html</code> , <code>server.py</code> , and the ICC ( <code>shell/web.py</code> ) all derive from it — bump that one file to release, never hard-code the version elsewhere. | 26.6       |
| <b>Real user data reaches the PUBLIC GitHub repo</b>                                                                       | The live data file lives OUTSIDE the repo; <code>.gitignore</code> is broad; a committed pre-push hook blocks any push containing data-shaped JSON; the app ships with no seed data.                                                            | 26.2, 26.3 |

| Trap (what bit us)              | Guard                                                                                                                                                                                                                                                                                                                        | Where      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| User data lost on an app update | <b>iPhone:</b> app code and your data are stored separately — an update replaces the code and never touches your on-device <code>localStorage/IndexedDB</code> (§22).<br><b>Computer / ICC:</b> every sync writes a rotating backup (last 10) to <code>data/backups/</code> , which is inside the ICC disaster-recovery tar. | §22 + 26.6 |

## 26.1 Why the repo is public (and must stay that way)

The iPhone PWA is served from GitHub Pages at <https://summerbb138.github.io/portfolio-tracker/>. On a **free GitHub plan**, **Pages serves only public repositories**. In mid-2026 the repo was switched to private; GitHub then **auto-disabled Pages**, the live URL began returning "Site not found," and iPhone updates silently stopped reaching the phone (the already-installed PWA kept running from its cache, so the breakage went unnoticed for weeks). Lesson: **do not make this repo private** while it hosts the PWA on the free plan. If privacy is ever required, the alternatives are a paid plan (private-repo Pages) or a different host — not simply flipping it private.

## 26.2 What keeps real data out of the repo

- **The live data file lives OUTSIDE the repo.** `pwa/server.py` reads and writes `~/Library/Application Support/PortfolioTracker/my-portfolio.json` — physically outside the git tree, so it cannot be committed even by accident. Portfolio data also lives in each device's browser `localStorage`. GitHub Pages does not serve it (verified: the data path returns 404 on the live site).
- **The app ships with no seed data.** New loads start with an empty portfolio (seed positions were removed in commit 5ad50e0 — see the history note below).
- **.gitignore is deliberately broad** so any stray data or backup copy is caught: it ignores every `my-portfolio` JSON file (any suffix), any filename containing `REAL-BACKUP`, the `data/backups/` directory, `setup.html`, `logs/`, `.claude/`, and `PROGRESS.md`.
- **A committed pre-push hook is the backstop** — `scripts/githooks/pre-push`, activated once per clone with `git config core.hooksPath scripts/githooks`. It aborts any push whose commits contain a JSON file shaped like portfolio data (a `positions` array of holdings carrying `costBasis`), independent of `.gitignore`, and refuses a push if `pwa/version.js` has lost its version. Tested to block a data-shaped file and pass clean changes.
- **Before every push:** the hook runs automatically; still, never `git add -f` a data file.

## 26.3 The clean-up performed on 2026-08-04 (repeat this recipe if real data ever lands in git)

Real positions had once been embedded as JavaScript seed data in early commits. The *working tree* was clean, but the *history* still held them — so making the repo public would have exposed them. Steps taken, in order:

1. **Backup real data outside the repo:** copied `data/my-portfolio.json` → `~/Desktop/Claude Summary/Portfolio-REAL-BACKUP-<timestamp>.json`, and bundled the full pre-cleanup git history →

~/Desktop/Claude Summary/portfolio-tracker-FULL-HISTORY-<timestamp>.bundle (so dev history isn't lost).

2. **Swap in throwaway data** while working: set data/my-portfolio.json to a single dummy holding (AAPL, no cash).

3. **Harden** .gitignore (the broad patterns in 26.2).

4. **Scrub doc examples:** genericized ticker examples to AAPL / Toyota (7203.T). (The doc PDF is regenerated cleanly from the Markdown — see 26.7.)

5. **Reset history to a single clean commit:**

```
git checkout --orphan clean-main
```

```
git add -A # respects .gitignore (no data/PDF)
```

```
git diff --cached --name-only # VERIFY: no my-portfolio.json, no PDF
```

```
git diff --cached | grep -iE "<real tickers>" # VERIFY: empty
```

```
git commit -m "Portfolio Tracker PWA v1.5 (build 25)"
```

```
git branch -M main
```

```
git grep -iE "<real tickers>" $(git rev-list main) # VERIFY: empty
```

```
git push --force origin main
```

6. **Only then make it public and enable Pages:**

```
gh repo edit summerbb138/portfolio-tracker --visibility public --accept-visibility-change-consequences
```

```
gh api -X POST repos/summerbb138/portfolio-tracker/pages -f 'source[branch]=main' -f 'source[path]='
```

7. **Restore real data locally** from the backup (gitignored, so it never reaches GitHub).

## 26.4 Residual risk & the bulletproof purge

`git push --force` makes the old commits *unreachable* on the branch, but **GitHub keeps them as dangling objects that are still fetchable by their exact 40-char SHA** (confirmed: the old seed-data commit responded to a public API call by SHA). Those SHAs are **not publicly discoverable** here (the commits were pushed while the repo was private; there are no forks, PRs, or public events exposing them), so practical exposure is very low — but it is not literally zero. To remove them **completely**, delete and recreate the repository (needs `delete_repo` permission), then push the clean tree and re-enable Pages. Until that is done, treat the pre-2026-08-04 commit SHAs as sensitive.

## 26.5 Deploy note — the root redirect

The app lives in `/pwa/` (the local server serves that directory), but GitHub Pages serves the **repo root**. A minimal root `index.html` redirects to `./pwa/` so the documented root URL resolves. Keep it; it contains no app code. `pwa/index.html` remains the single source of truth. The legacy root `sw.js` / `manifest.json` / `icons` are dormant migration shims for browsers that installed the old root-scoped app before it moved to `/pwa/` — leave

them so those installs migrate cleanly.

## 26.6 Foolproofing (added 2026-08-04) — one version source, data location, backups, hook

Three mechanisms keep the standing risks from recurring:

**(1) One version source** — `pwa/version.js`. It sets `APP_VERSION` and `APP_BUILD` and is the **ONLY** place the version is written. Everything derives from it: `pwa/sw.js` does `importScripts` on it and builds its cache name from `APP_BUILD` (so bumping the file changes the cache name **AND** triggers the service-worker update); `pwa/index.html` stamps its badge from it; `pwa/server.py` and the ICC shell (`shell/web.py`, `_pwa_version()`) parse it. To release, bump `version.js` only — never hard-code the version anywhere else.

**(2) Live data outside the repo + (3) rotating backups inside it.** The live file is external (26.2). On every sync, `server.py` writes a timestamped copy to `data/backups/` and keeps the last 10. That directory is gitignored (never on GitHub) but sits inside `~/Desktop/Claude Summary`, so the ICC disaster-recovery tar captures it — the data stays recoverable without being exposed. The pre-push hook (26.2) is the commit backstop.

## 26.7 Deviations from the house standards (recorded per **PROJECT\_STANDARDS**)

- **Data location:** the standard keeps data in the project's `data/` folder (gitignored). Because this repo is PUBLIC, the live data file is deliberately moved **OUTSIDE** the repo (26.2); its rotating backups stay under `data/backups/` so they remain inside the disaster-recovery backup. Approved trade-off: maximum commit-safety while staying recoverable.
- **PDF generator:** the retired `generate_pdf.py` is gone; this doc's PDF is now produced by the shared `Project Template/docs/md_to_pdf.py`. Keep the prose free of bare asterisks — that converter mis-parses a literal asterisk even inside a code span.